软 件 工 程

实验二：软件设计与建模

王睿阳 241880548

2026 年 11 月 1 日

项目：外卖点餐与配送系统（校园食光）

目录

一、项目与选题说明 1

二、软件的主要功能 1

三、软件架构与 UML 建模 2

1 模块划分与依赖：组件图 2

2 领域模型：类图 4

3 角色与用例：用例图 6

4 订单状态机：状态图 7

5 端到端流程：活动图 8

6 交互细节：时序图 9

四、UI 界面设计 10

1 登录与角色入口 11

2 顾客端：店铺列表、店铺详情、确认订单、订单跟踪 12

3 商家端：订单工作台、菜品与库存 14

4 骑手端：任务列表、配送异常上报 15

5 管理端：入驻审核、订单干预与仲裁 16

五、AI 辅助设计过程 17

1 记录一：组件图分层 17

2 记录二：用例图拆分 17

3 记录三：活动图简化 17

4 记录四：UI 原型调整 18

5 一次通过的三张图 18

6 对大模型辅助软件设计的思考 18

六、版本信息与交付说明 19

一、项目与选题说明

本项目延续实验一的需求分析成果，选题为「3. 外卖点餐与配送系统」，面向一所高校校内的点餐与配送场景：学生在宿舍点餐，校内及周边商户出餐，平台自营骑手配送，平台运营方负责审核与兜底处置。

一期范围是一个校区的宿舍片区，二十余家商户；不接入真实支付，只使用钱包余额；不接入第三方地图，收货地址从预设楼栋门牌列表中选择；不做菜品规格与加料、不做评价与优惠券、不做提现与真实资金结算。这些边界在实验一已经与需求方确认，并在本实验的类图、用例图和 UI 中保持一致——需求中标记为「已拒绝」的两条（订单评价、优惠券）在设计中不出现对应类与界面。

本实验的目标不是写代码，而是把需求转化成可实施的软件结构：模块怎么切、类怎么建、状态怎么流转、界面长什么样，为实验三的实现留下清晰的施工图。

二、软件的主要功能

系统按职责划分为九个模块，与实验一的需求编号一一对应，便于追溯：

模块	主要功能	对应需求
用户与权限	四类角色登录与鉴权、封禁、角色数据隔离	REQ-001
商家与店铺	入驻申请与审核、营业中 / 休息中 / 平台关闭三种状态	REQ-002、REQ-003
商品与库存	菜品增删改与上下架、每日份数、预占与释放、售罄、搜索排序	REQ-004 ~ REQ-006
购物车与订单	单店购物车、下单校验、价格快照、提交令牌幂等、状态机	REQ-007 ~ REQ-009、REQ-024
支付与退款	余额支付、整单退款、部分退款、金额一致性	REQ-010、REQ-014、REQ-022
配送调度	按片区派单、骑手接单与拒单、改派、送达、配送异常	REQ-012、REQ-013、REQ-016
售后与仲裁	取消规则、商家同意退款、超时自动同意、管理员仲裁	REQ-014、REQ-015、REQ-026
审计与通知	关键操作留痕、状态变更记录、站内通知	REQ-017
运维支撑	超时任务调度、每日库存重置、查询性能与可用性	REQ-020、REQ-021、REQ-023

其中「运维支撑」不是面向用户的界面模块，而是由定时任务和基础设施承担，体现在组件图的 Scheduler 与 UI 之外的部分。

三、软件架构与 UML 建模

本实验共绘制 8 张 UML 图，覆盖题目要求的三类：结构性图（类图、组件图）、行为性图（用例图两张、状态图、活动图）、交互性图（时序图两张）。

1 模块划分与依赖：组件图

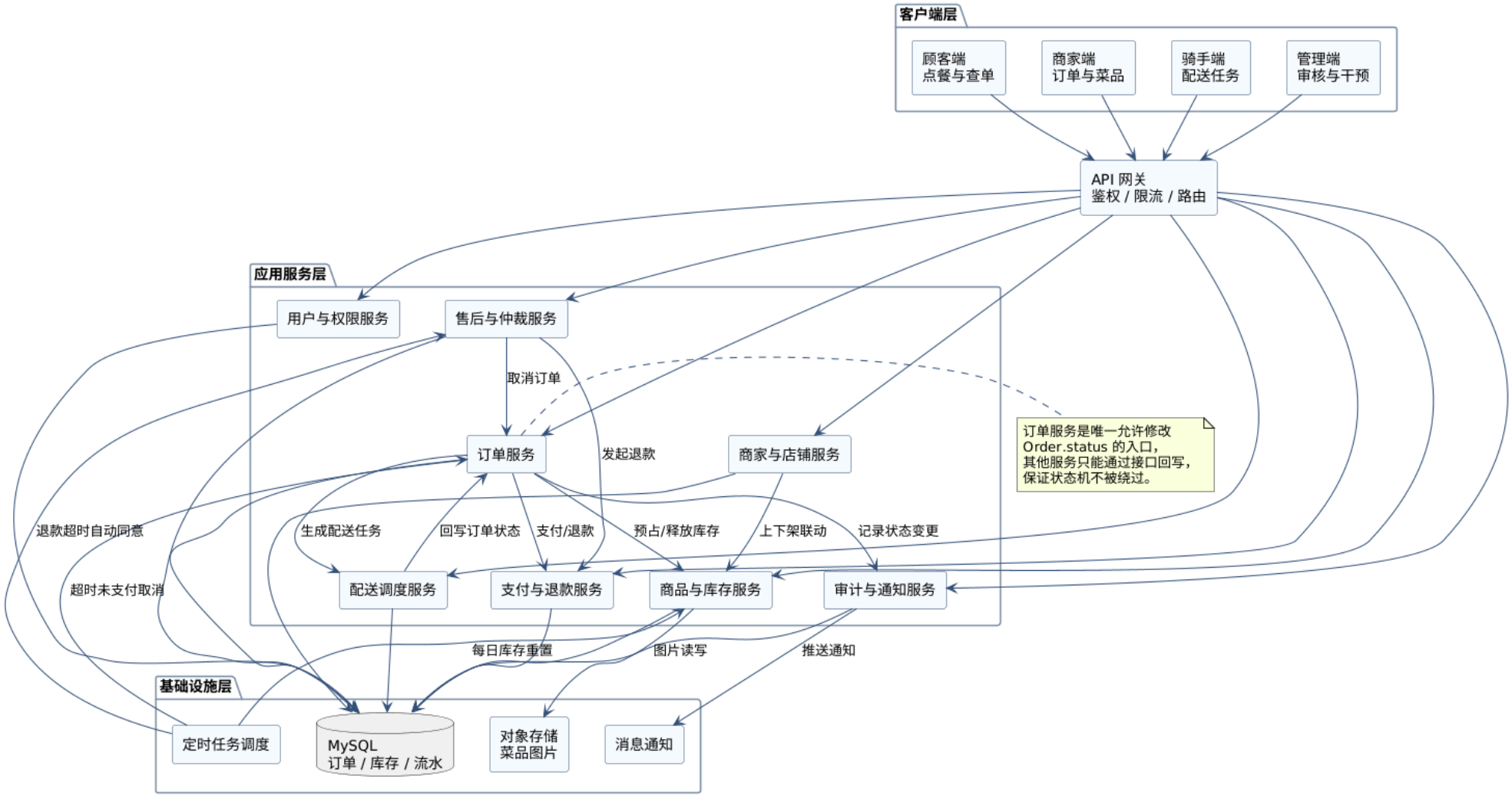
组件图按三层组织。**客户端层**是四套前端：顾客端（点餐与查单）、商家端（订单与菜品）、骑手端（配送任务）、管理端（审核与干预）。**应用服务层**是八个领域服务，与上表模块一一对应。**基础设施层**是 MySQL、定时任务、对象存储与消息通知。

这样切的理由有三点：

第一，**按业务能力而不是按页面切。**如果按"顾客端后台、商家端后台"切，订单状态会被两套代码同时修改，实验一里"状态一致"的要求就守不住。现在订单状态只有一个入口——订单服务，其他服务通过接口回写，这一点在图上用注释标了出来。

第二，**加一个 API 网关。**第一版组件图是客户端直接连八个服务，三十多条线在图上缠成一团，且鉴权逻辑要重复八遍。改成客户端统一走网关后，鉴权、限流、路由收敛到一处，客户端与服务之间变成 4 条入边加 8 条出边，图也读得懂了。

第三，**把定时任务单独拿出来。**超时未支付取消、十分钟未接单取消、退款申请二十四小时自动同意、每日零点库存重置，这四件事都不能靠用户操作触发，必须由调度器驱动。如果不单独成组件，这些规则在类图里会找不到落点。


客户端层
顾客端
点餐与查单
商家端
订单与菜品
骑手端
配送任务
管理端
审核与干预
API 网关
鉴权 / 限流 / 路由
应用服务层
用户与权限服务
售后与仲裁服务
取消订单
订单服务
发起退款
商家与店铺服务
订单服务是唯一允许修改
Order.status 的入口，
其他服务只能通过接口回写，
保证状态机不被绕过。
退款超时自动同意
生成配送任务
回写订单状态
支付/退款
预占/释放库存
上下架联动
记录状态变更
超时未支付取消
配送调度服务
支付与退款服务
商品与库存服务
审计与通知服务
每日库存重置
图片读写
推送通知
基础设施层
定时任务调度
MySQL
订单 / 库存 / 流水
对象存储
菜品图片
消息通知

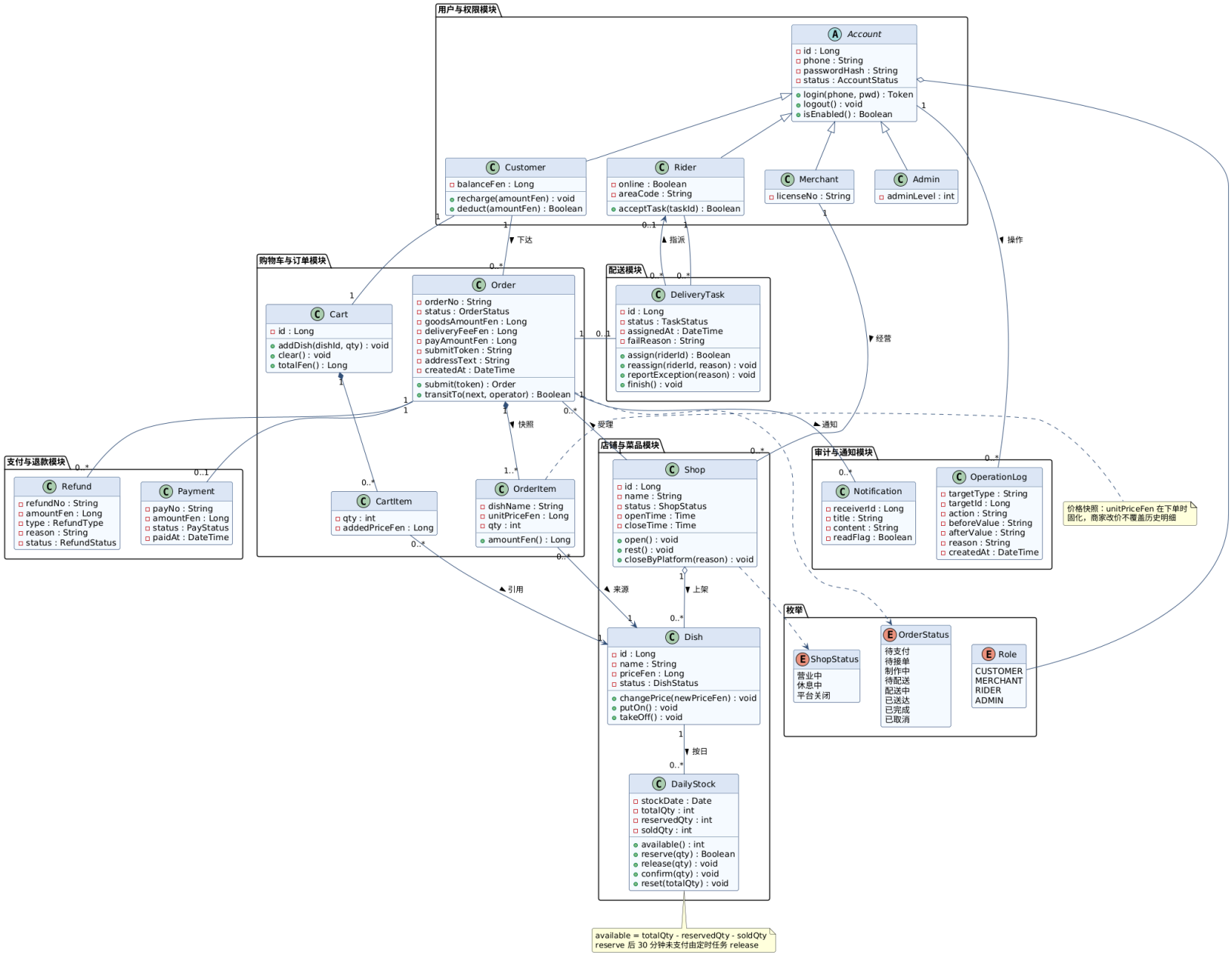
UML组件图

2 领域模型：类图

类图按六个包组织：用户与权限、店铺与菜品、购物车与订单、支付与退款、配送、审计与通知，另有一个枚举包。

几个关键设计点：

- **Account 是抽象父类**，Customer / Merchant / Rider / Admin 继承它，role 字段配合权限判断。商家与店铺是 1:1，不做店员子账号，与 REQ-001 一致。
- **DailyStock 单独成类**，按「菜品 + 日期」记录 totalQty、reservedQty、soldQty。可用份数等于三者相减，下单时 reserve、超时 release、支付成功后 confirm。如果只把库存做成 Dish 上的一个数字，就没法表达"预占"和"已售"的区别，也解释不了 REQ-005 的并发要求。
- **OrderItem 保存快照。**dishName、unitPriceFen、qty 是冗余字段，正是为了保证商家改价不影响历史订单（REQ-009）。这是一处有意的冗余。
- **Order 上带 submitToken。**同一个令牌重复提交只会命中同一笔订单（REQ-024），不需要额外的去重表。
- **金额统一用 Long 存分**，不用浮点，对应 REQ-022 与约束 REQ-027。


用户与权限模块
Account
id : Long
phone : String
passwordHash : String
status : AccountStatus
login(phone, pwd) : Token
logout() : void
isEnabled() : Boolean
Customer
balanceFen : Long
recharge(amountFen) : void
deduct(amountFen) : Boolean
Rider
online : Boolean
areaCode : String
acceptTask(taskId) : Boolean
Merchant
licenseNo : String
Admin
adminLevel : int
下达
指派
操作
经营
购物车与订单模块
Cart
id : Long
addDish(dishId, qty) : void
clear() : void
totalFen() : Long
Order
orderNo : String
status : OrderStatus
goodsAmountFen : Long
deliveryFeeFen : Long
payAmountFen : Long
submitToken : String
addressText : String
createdAt : DateTime
submit(token) : Order
transitTo(next, operator) : Boolean
配送模块
DeliveryTask
id : Long
status : TaskStatus
assignedAt : DateTime
failReason : String
assign(riderId) : Boolean
reassign(riderId, reason) : void
reportException(reason) : void
finish() : void
快照
受理
通知
支付与退款模块
Refund
refundNo : String
amountFen : Long
type : RefundType
reason : String
status : RefundStatus
Payment
payNo : String
amountFen : Long
status : PayStatus
paidAt : DateTime
CartItem
qty : int
addedPriceFen : Long
OrderItem
dishName : String
unitPriceFen : Long
qty : int
amountFen() : Long
引用
来源
店铺与菜品模块
Shop
id : Long
name : String
status : ShopStatus
openTime : Time
closeTime : Time
open() : void
rest() : void
closeByPlatform(reason) : void
上架
Dish
id : Long
name : String
priceFen : Long
status : DishStatus
changePrice(newPriceFen) : void
putOn() : void
takeOff() : void
按日
DailyStock
stockDate : Date
totalQty : int
reservedQty : int
soldQty : int
available() : int
reserve(qty) : Boolean
release(qty) : void
confirm(qty) : void
reset(totalQty) : void
available = totalQty - reservedQty - soldQty
reserve 后 30 分钟未支付由定时任务 release
审计与通知模块
Notification
receiverId : Long
title : String
content : String
readFlag : Boolean
OperationLog
targetType : String
targetId : Long
action : String
beforeValue : String
afterValue : String
reason : String
createdAt : DateTime
价格快照：unitPriceFen 在下单时固化，商家改价不覆盖历史明细
枚举
ShopStatus
营业中
休息中
平台关闭
OrderStatus
待支付
待接单
制作中
待配送
配送中
已送达
已完成
已取消
Role
CUSTOMER
MERCHANT
RIDER
ADMIN

UML类图

3 角色与用例：用例图

用例图拆成两张。第一版是单张图把 25 个用例塞进去，导出的图 4096×452 像素，缩到 A4 宽度后字小到看不清，而且不同角色的用例混在一起，关系线交叉严重。拆成"顾客与商家侧"和"履约与管理侧"之后，每张图都能单独放进一页。

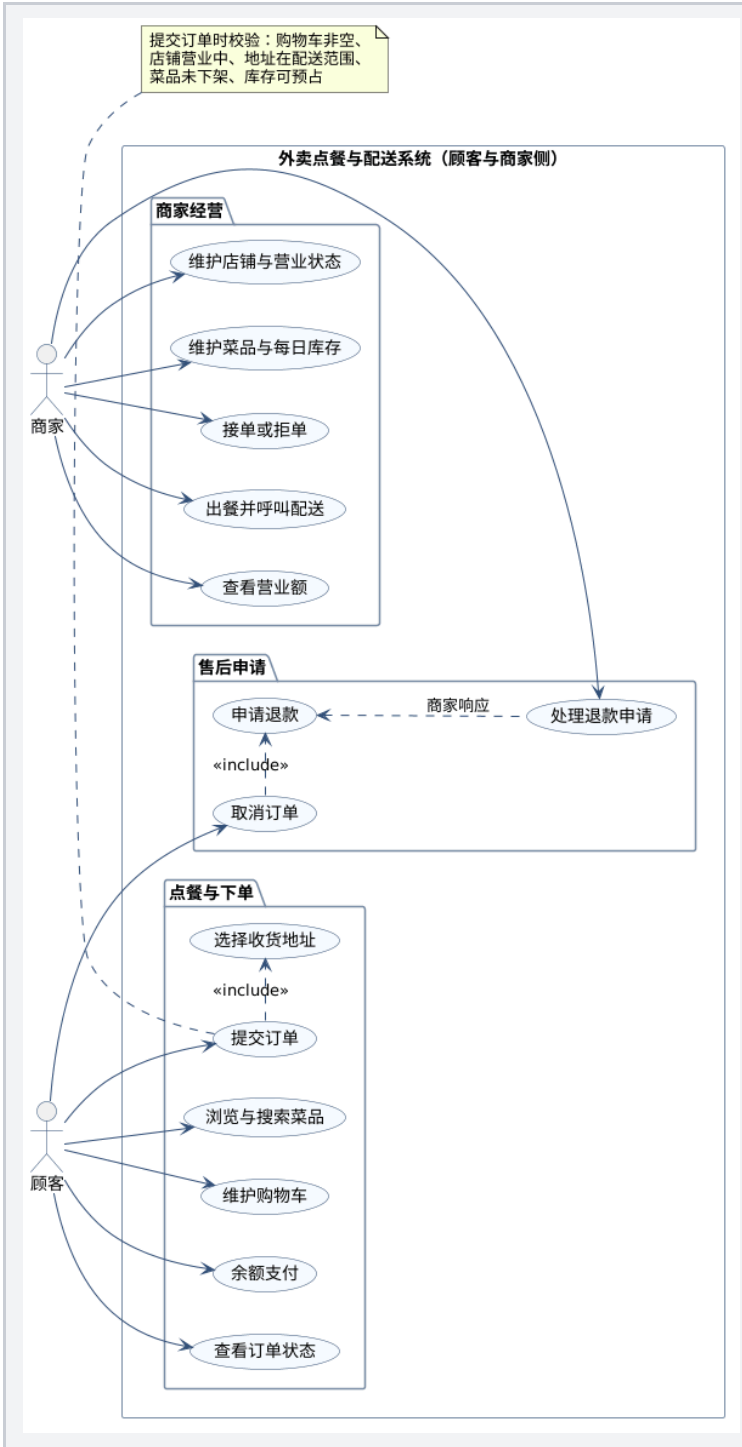


图 3-3 顾客与商家侧用例图

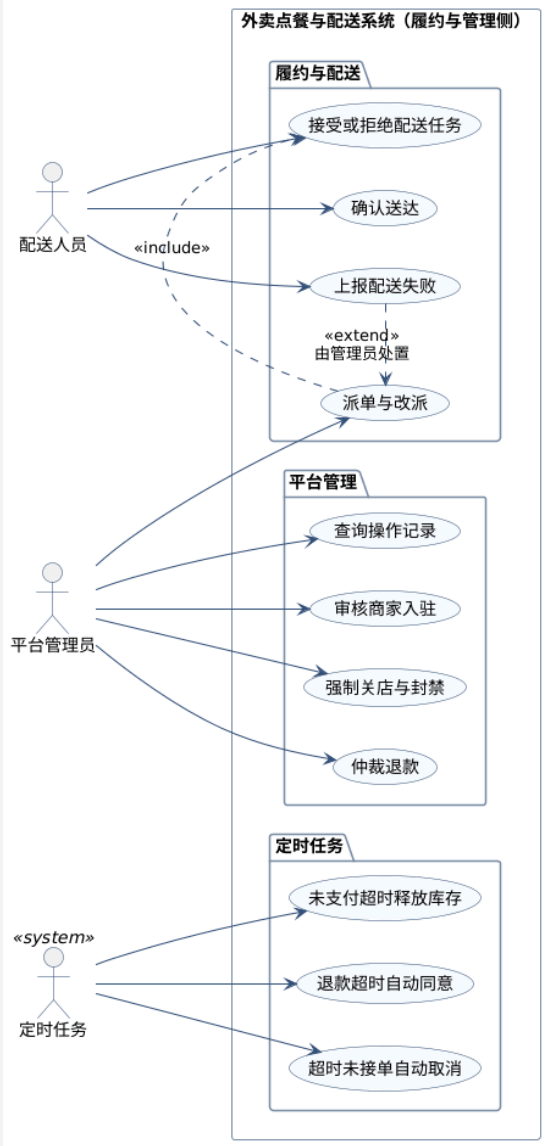


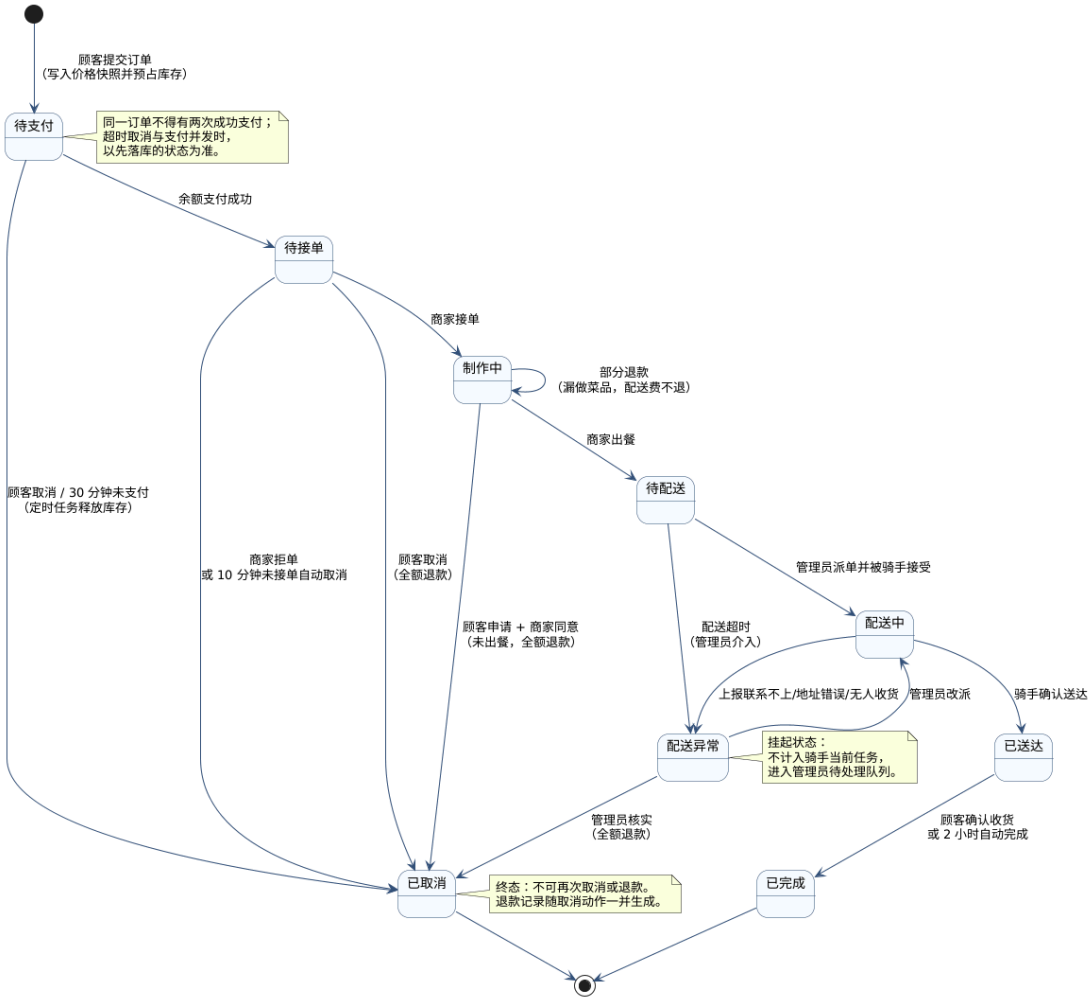
图 3-4 履约与管理侧用例图

第一张覆盖点餐主链路：顾客浏览、加购、提交订单、支付、查单、取消；商家维护店铺与菜品、设置库存、接单或拒单、出餐、处理退款、看营业额。第二张覆盖履约与管理：骑手接受任务、确认送达、上报异常；管

理员派单改派、审核入驻、强制关店封禁、仲裁退款、查操作记录；外加一个系统角色"定时任务"，代表三条自动规则。

图上只保留了三条真实关系：提交订单 include 选择收货地址、取消订单 include 申请退款、上报配送失败 extend 派单改派。初版把"接单 include 出餐""派单 include 接受任务"也画上了，其实那是先后顺序而不是包含关系，属于关系用错，已经删掉。

4 订单状态机：状态图



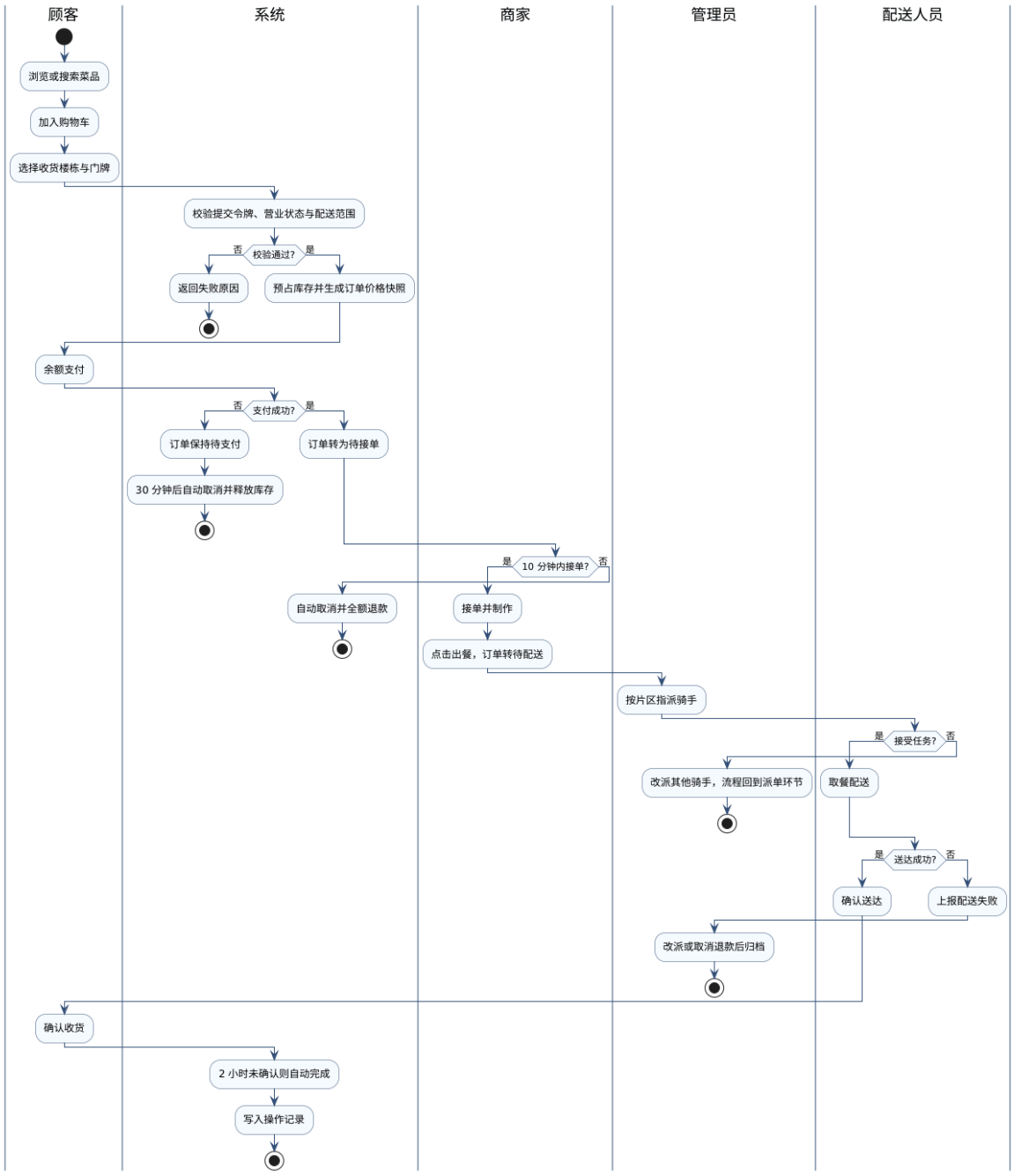
UML状态图

订单有八个状态：待支付、待接单、制作中、待配送、配送中、已送达、已完成、已取消，另有配送异常作为挂起状态。

状态图要回答的是"谁能把订单推到哪一步"，所以每条迁移都标了触发方和条件：支付成功才进待接单；商家十分钟不接单自动取消；顾客接单后取消必须商家同意且未出餐；出餐后不可取消，只能按菜品部分退款；配送失败进入配送异常，由管理员改派或取消退款；已送达两小时顾客不确认则自动完成。

两个刻意的约束：**已完成和已取消是终态**，不能再退款，否则对账会乱；**部分退款是自环**，不改变订单状态，只增加退款记录。这一点在初稿里没写，是画状态图时才发现"部分退款到底算不算状态"必须有个明确答案。

5 端到端流程：活动图



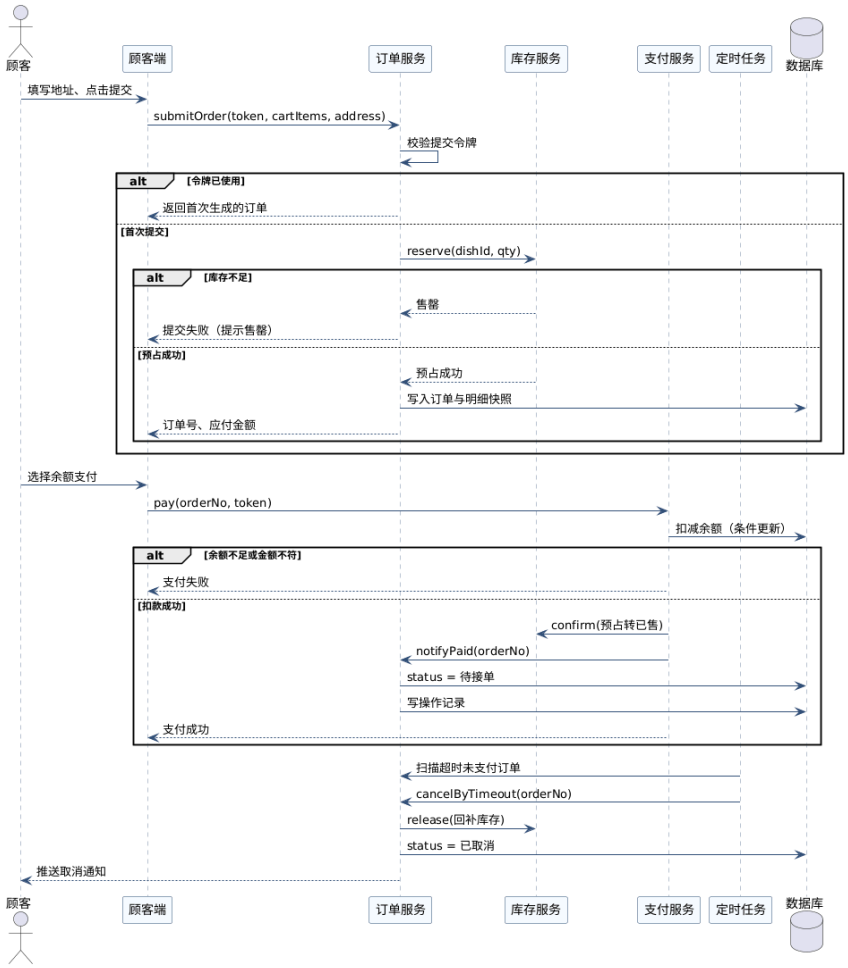
UML活动图

活动图用五条泳道（顾客、系统、商家、管理员、配送人员）串起一笔订单的完整生命周期，重点在分支：校验失败直接终止、支付超时终止并释放库存、商家超时不接单终止并退款、骑手拒单回到派单、配送失败由管理员处置后归档。

所有分支都必须有明确终点，是这张图反复改的地方。初版在配送失败处套了三层判断，出现跨泳道嵌套，导出后连线交叉、出现两个并列的结束节点，读图的人根本分不清哪条路通向哪。最后把"改派还是取消"合并成管理员的一个动作节点，细节交给状态图和时序图表达——**一张图只讲一件事，不要既画流程又画规则。**

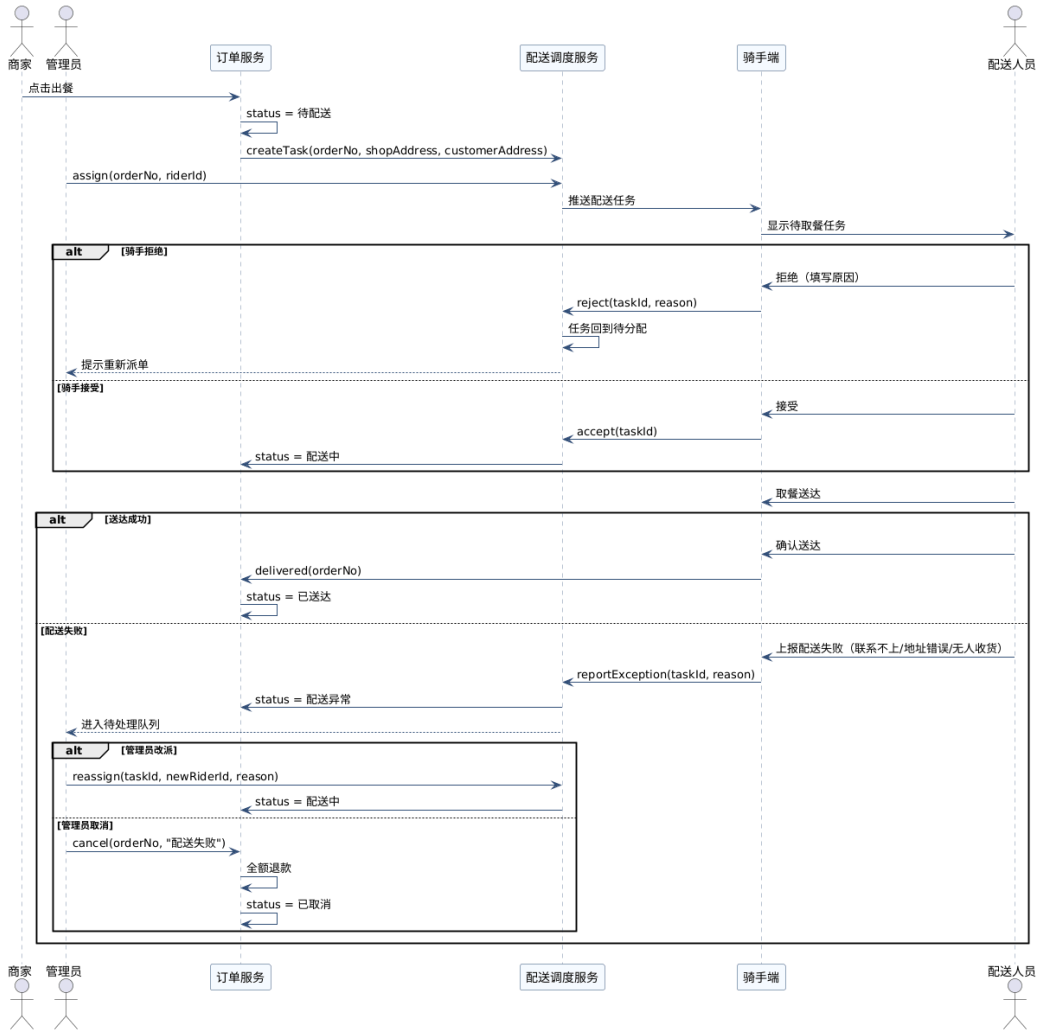
6 交互细节：时序图

下单与支付这张图重点在三个时序陷阱：令牌幂等（重复提交返回首次结果）、库存预占与回补（支付失败或超时释放）、扣款与订单状态的先后。



UML时序图-下单与支付

派单与配送这张图重点在分支归属：骑手拒单回到待分配，配送异常进入管理员队列，管理员可以改派也可以取消退款。



UML时序图-派单与配送异常

两张图都用了 alt 组合片段表示分支，参与者只保留真正参与该场景的对象，数据库单独画出来是因为"状态最终落在哪张表"是实现时最容易含糊的地方。

四、UI 界面设计

界面按四个角色的使用场景设计，共 11 屏，覆盖九个模块。工具上使用 HTML/CSS 编写高保真原型后截图导出（题目允许"亿图、figma、pixso 等"工具，HTML 原型同样可以精确控制布局与配色，且便于后续直接转成前端页面）。视觉上统一用橙色 #FF6B2C 作为主色，灰色 #F5F6F8 作为背景，圆角 12px，移动端 390×844，桌面端 1440×900。

1 登录与角色入口



UI-01

一屏解决两件事：登录，以及告诉用户"这套系统有四种身份"。下方用标签列出四类角色并注明"登录后按角色进入对应端"，右侧说明封禁与子账号规则。设计上把权限说明放在登录页，是因为实验一里"账号被封禁后任何角色都不能登录"是明确需求，用户需要在入口处就知道为什么登不上。

2 顾客端：店铺列表、店铺详情、确认订单、订单跟踪



图 4-2 店铺列表：卡片承载营业状态，休息中店铺置灰但仍可浏览



图 4-3 店铺详情：分类 + 菜品，行内显示剩余份数与售罄标记



图 4-4 确认订单：地址范围、价格锁定、支付方式三项前置



图 4-5 订单跟踪：状态时间轴与接单倒计时

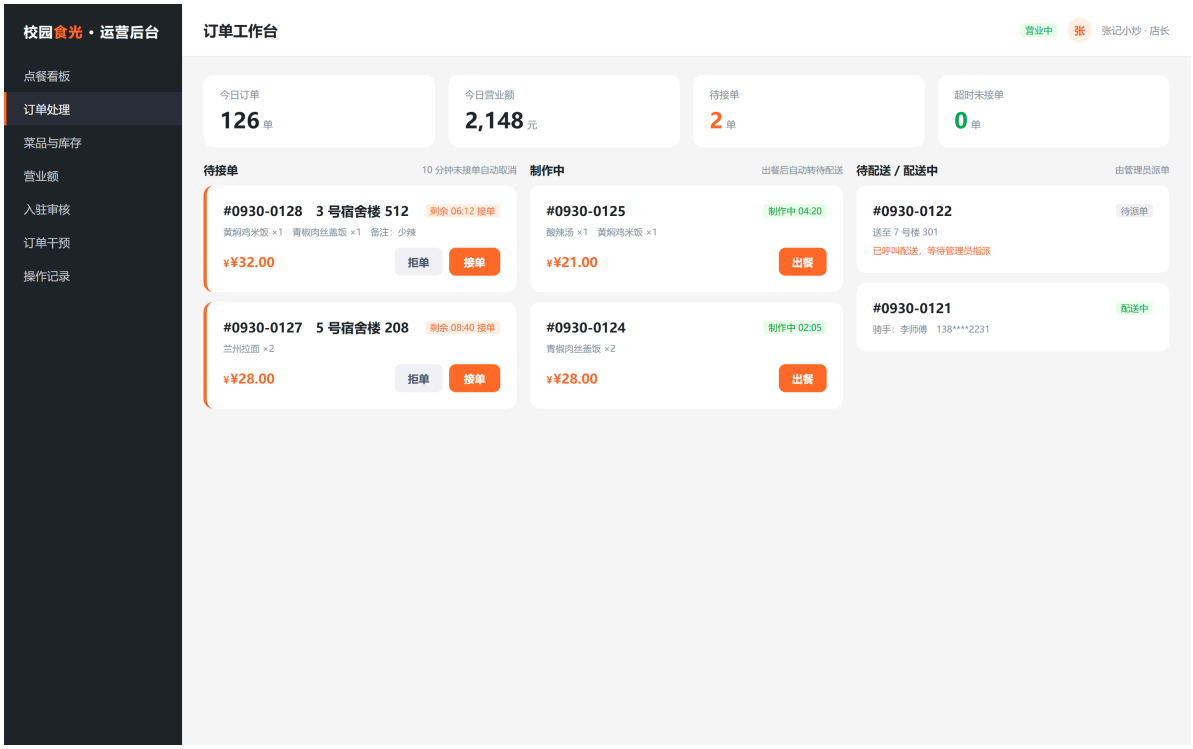
店铺列表用卡片承载"营业中 / 休息中"状态：休息中的店铺置灰但仍可浏览，只有下单入口被禁用，对应 REQ-003。搜索框放在顶部，排序项（销量、距离、只看营业中）直接露出，避免再进一层筛选页。

店铺详情采用左侧分类、右侧菜品的经典结构。菜品行里同时显示月售和**剩余份数**，售罄的菜品整行置灰并打上"售罄"标签，但不下架——这样顾客能看到"今天卖完了"，而不是以为这家店没有这道菜。底部购物车条常驻，合计金额与配送费分开显示。

确认订单页把三个容易出错的信息前置：地址是否在配送范围内、价格是否已锁定、支付方式与余额。页面底部写明"提交即预占库存并锁定价格，30 分钟未支付自动取消"，把系统行为讲在用户点按钮之前，减少后续争议。

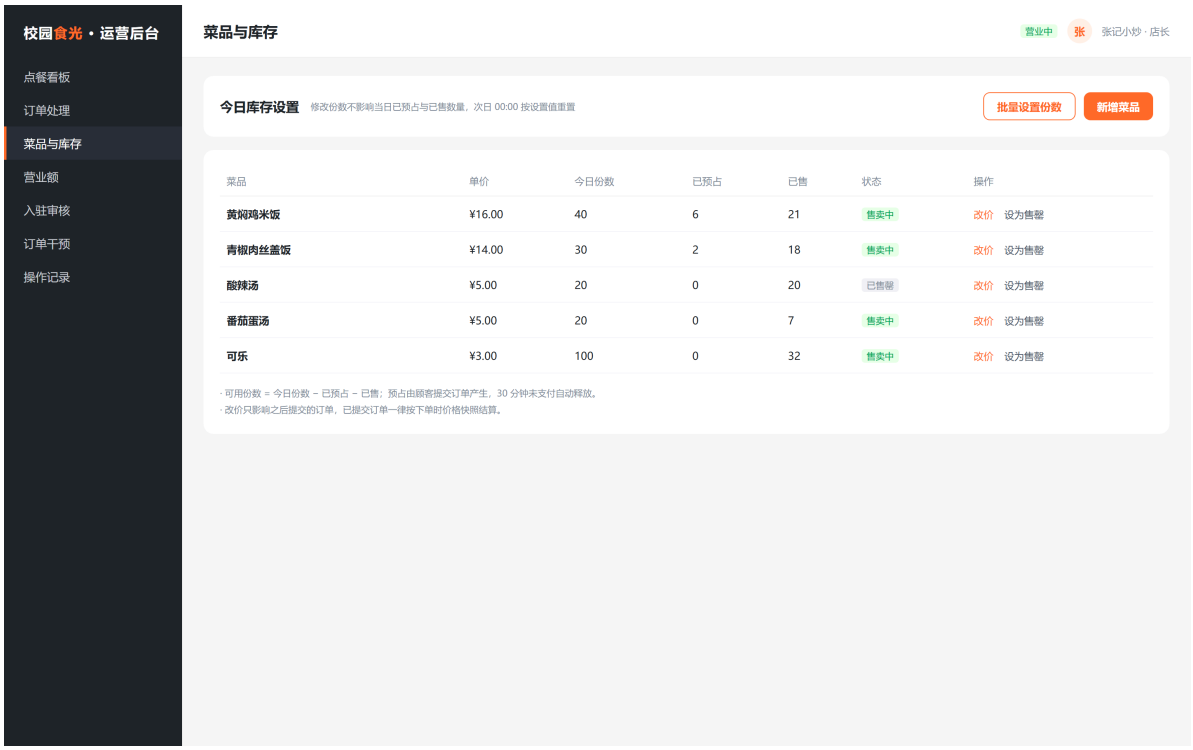
订单跟踪页用竖向时间轴展示状态推进，顶部直接给出"09:32 前未接单将自动取消"的倒计时。这类倒计时在实验一里是"十分钟未接单自动取消"的落地形式，如果不显示，顾客会以为订单卡住了。

3 商家端：订单工作台、菜品与库存



UI-06

订单工作台按"待接单 / 制作中 / 待配送配送中"三列排布，对应订单状态机的前半段。待接单卡片上带剩余秒数，接单与拒单并排；制作中的卡片只有"出餐"一个动作，避免误操作。顶部四个指标里特意放了"超时未接单"，因为这是会被平台统计的负向指标。



UI-07

菜品与库存页的核心是把"今日份数 / 已预占 / 已售"拆成三列。很多同类系统只显示"剩余"，商家看不懂为什么明明设了 40 份却只能卖 20 份——因为 6 份被未支付的订单占着。页面底部用两行小字把这条规则和"改价只影响之后的订单"写清楚。

4 骑手端：任务列表、配送异常上报



图 4-8 骑手任务列表：今日数据 + 待取 / 配送中任务



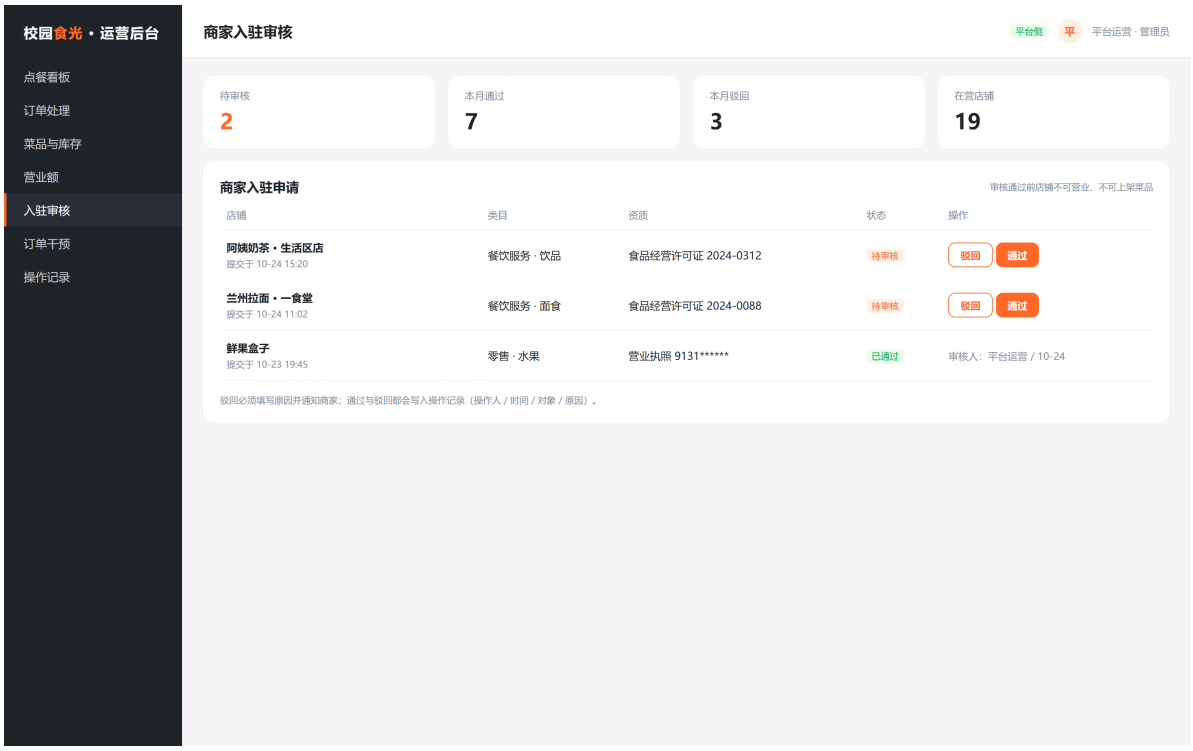
图 4-9 配送异常上报：原因限定三选一

骑手端首屏只放两件事：今天跑了多少、现在该干什么。任务卡片显示取餐点、送达点、剩余距离和配送费，接受与拒绝并排，拒绝是允许的——实验一里明确"骑手可以不接，不接就换人"。

异常上报把原因限定为三种：联系不上顾客、地址错误、无人收货。限定选项而不是让骑手自由填写，是为了让后台能按原因统计、按原因决定处置方式。提交后订单挂起，页面明确告知"不再计入当前任务，由管理员

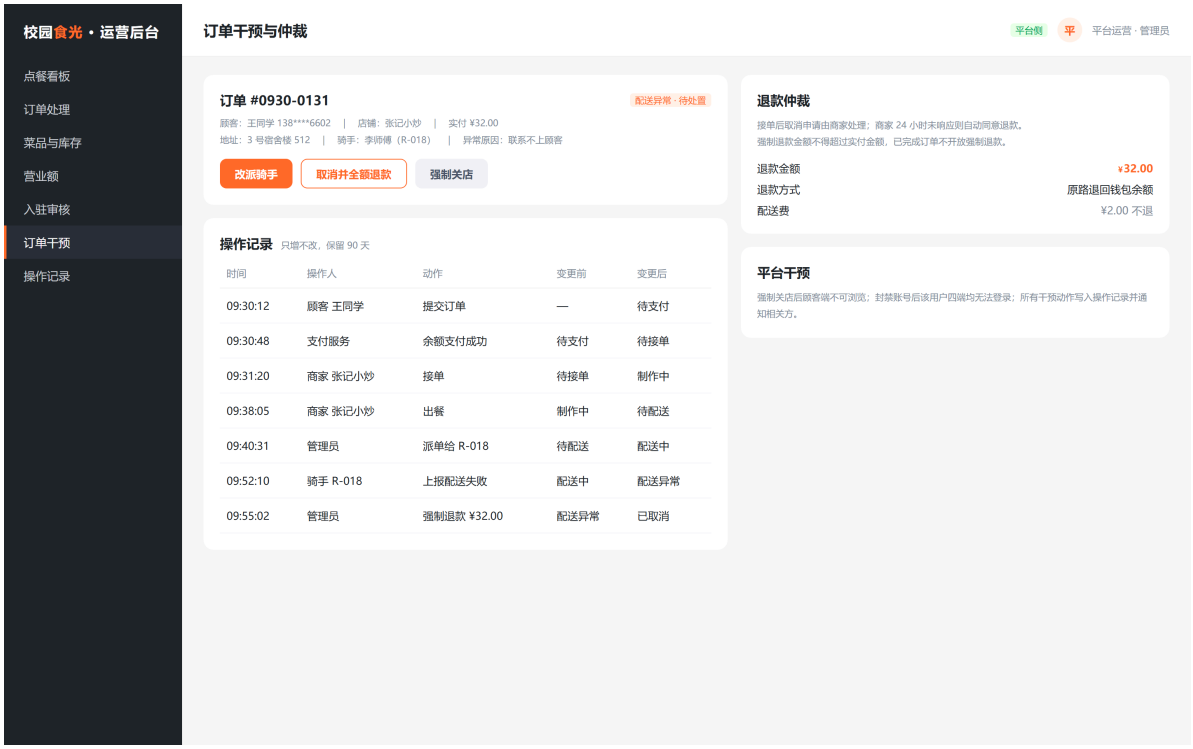
处置"。

5 管理端：入驻审核、订单干预与仲裁



UI-10

入驻审核列表直接给出店铺、类目、资质、状态与两个动作。驳回必须填原因、通过与驳回都要留痕，这两点在页面底部注明，对应 REQ-002 与 REQ-017。



UI-11

订单干预页把订单信息、异常原因、三个处置动作（改派、取消退款、强制关店）和完整的操作记录放在同一屏。操作记录用表格展示"时间 / 操作人 / 动作 / 变更前 / 变更后"，这正是审计模块要落库的字段；把它画在界面上，实现时不容易漏字段。

五、AI 辅助设计过程

本实验使用 OpenCode（CLI 客户端）内置 Agent，模型为 DeepSeek V4.1 Flash。分工是：我负责确定模块边界、状态规则和每个界面要承载哪些信息；模型负责把已经想清楚的内容写成 PlantUML 源码、HTML/CSS 原型和文档章节。UML 图片由 PlantUML 渲染服务出图，UI 由无头浏览器截图导出。下面按实际发生顺序记录五次主要使用，以及三张一次通过、未做内容修改的图。

1 记录一：组件图分层

使用场景：把实验一的九个模块交给模型，要求按分层架构给出组件划分与依赖关系。

生成结果：客户端四个端直接连到八个应用服务，服务之间再互相调用，全图约三十条依赖线，没有统一的入口层。

优缺点分析：优点是模块职责切得准，服务之间的调用方向也对（订单服务调库存、调支付、生成配送任务）；缺点是只考虑了"谁能调谁"，没考虑鉴权放在哪里，也没考虑图上能不能读——渲染出来是一张 2000×740 的网，线条在中间缠成一团。

修改优化后的结果：加入 API 网关收口，客户端只连网关，鉴权、限流、路由收敛到一处；服务间依赖从十几条砍到八条；补上"订单服务是订单状态唯一写入口"的注释。第二版仍有几条长线穿过图心，第三版把基础设施层的连线归并，最终 1506×779，能放进一页。

2 记录二：用例图拆分

使用场景：按四类角色列出用例，并标注 include / extend 关系。

生成结果：一张图里放了 25 个用例，纵向排列，导出尺寸 457×2000。

优缺点分析：优点是覆盖完整，取消、退款、仲裁这些次要流程都列出来了；缺点是完全没有版面意识。这个尺寸缩到 A4 宽度后字号不到 3pt，等于没画。另外它把"接单 include 出餐""派单 include 接受任务"也写成了包含关系，这两条其实是先后顺序，不是必然发生的子步骤。

修改优化后的结果：先改成横向排列，结果变成 4096×452，更没法看；最后按角色拆成两张——"顾客与商家侧"和"履约与管理侧"，每张约 10 个用例，尺寸回到 590×1168 和 453×949。同时删掉两条错误的包含关系，只保留提交订单 include 选择地址、取消订单 include 申请退款、上报异常 extend 派单改派。

3 记录三：活动图简化

使用场景：用五条泳道画出一笔订单从下单到完成的端到端流程。

生成结果：主干流程正确，但在"配送失败"处套了三层判断（送达成功？→ 重新派单？→ …），跨泳道嵌套。

优缺点分析：优点是分支考虑得很全，连骑手拒单后回到派单这种回路都画了；缺点是把"流程"和"规则"混在一起——重新派单属于处置规则，图里却画成了主流程的一个分叉。导出后连线横穿整个图，还出现了两个并列的结束节点，读图的人分不清哪条路通向哪。

修改优化后的结果：砍掉二级嵌套，把"改派还是取消退款"合并为管理员的一个动作节点，细节交给状态图和时序图表达。原则定为一张图只讲一件事：活动图讲流程，状态图讲规则。

4 记录四：UI 原型调整

使用场景：让模型按模块生成 HTML/CSS 高保真原型，覆盖四个角色共 11 屏。

生成结果：结构合理、风格统一，但有三处明显问题：管理端顶栏沿用了"张记小炒 · 店长"，身份错位；店铺详情页菜品太少，下半屏大片留白；商家库存页只写了"剩余"，看不出预占和已售的差别。

优缺点分析：优点是出图快、样式统一，改一版只要调几个变量；缺点是它不会质疑自己给的情境——管理端顶着商家的身份、残留着别的页面里的角色设定，它不会主动发现，也不会问"这个数对商家有没有意义"。

修改优化后的结果：桌面端模板增加角色参数，管理端改为"平台运营 · 管理员"；店铺详情补三个菜品消除留白；库存页拆成"今日份数 / 已预占 / 已售"三列，并在页脚用两句话解释为什么设了 40 份却只能卖 20 份。

5 一次通过的三张图

类图、状态图、时序图基本一次成型，没有做内容修改，只调整了观感：类图给了包配色和阴影关闭；状态图给"配送异常"加了挂起说明；时序图指定了 `alt` 片段的分支名。这三张之所以顺，是因为需求里状态规则、金额口径、边界条件在实验一已经写死，模型只需要翻译，不需要判断。反过来，凡是需要它替我做取舍的（模块怎么切、图放多少信息、界面给谁看），它都出了问题——这条规律在下面总结里还会提到。

6 对大模型辅助软件设计的思考

这一轮下来，模型最合适的位置不是"设计者"而是"打字员"：它把已经想清楚的东西快速变成 PlantUML 源码、HTML 结构和文档章节，速度快、格式规范；但它不负责判断"该不该有这个功能""这张图该放多少信息""这个界面是给谁看的"。三次返工（组件图、用例图、活动图）都不是语法错误，而是尺度判断错误——线条多到没法读、图的比例不适合排版、流程和规则混在一张图里。

由此可以设想一条工作流：**人定边界 → 模型出草稿 → 规则自动校验 → 人审语义 → 模型改版。**规则校验这一层可以自动化，比如用脚本检查每个图是否只包含一类元素、每个用例是否都能追溯到实验一的需求编号、状态图与时序图的分支是否指向同一个处置结果；排版也可以自动，比如导出后按像素判断长宽比是否落在可排版的区间内。这一层能过滤掉本次遇到的大部分低级问题。

但"人只做检查校对"这件事，我认为折扣很大。检查本身是有成本的：本次三次返工，判断"这张图不能这么画"靠的是我看到渲染结果后的直觉，而不是一条能写进脚本的规则。真正能省下来的，是画图和改图的时间；省不下来的，是"该画什么"的判断。所以更现实的分工是：人决定边界与取舍，模型承担实现与返工，规则脚本兜住低级错误——人从"画图的人"变成"定标准的人"，而不是变成"只看一眼的人"。

六、版本信息与交付说明

仓库地址：https://github.com/marcus-37/lab1-takeout

分支名称：main

本实验在仓库中的位置：lab2/ 子目录（实验一的访谈、需求与报告在仓库根目录，两组内容互不覆盖）

仓库提交顺序：第一次提交为 AI 生成的初稿（组件图无网关、用例图为单张 25 个用例、活动图带跨泳道嵌套）；第二次提交为人工审查后的修改与实验报告。两次提交的差异就是本次人工修改的记录。

交付目录结构：

路径	内容
实验2_241880548_王睿阳/实验报告.pdf	本报告
实验2_241880548_王睿阳/UML/	8 张 UML 图的 .puml 源码与 .png 导出图
实验2_241880548_王睿阳/UI界面/	11 张界面设计稿，按角色与模块编号
build/	生成用的脚本（UML 渲染、界面截图、报告排版），不属于提交内容

UML 图与界面均可复现：修改 .puml 后运行 build/render_puml.py 重新出图；修改 build/ui_gen.py 后运行 build/ui_shot.py 重新截图；报告由 report.md 经 build/md2html2.py 转 HTML 后导出 PDF。

关于设计与实现的关系：本实验只做设计不做实现，状态机、类图字段与界面上的规则说明都按"后续可以直接照着写代码"的标准标注，例如订单服务是唯一写入口、金额以分为单位存储、库存按日预占。实验三实现时如果与设计有出入，以实现为准回改设计，但不影响本报告的完整性。